Jinhee Mok

AI & Automation Engineer

Career Description

jinheemok815@gmail.com
+82 10-9030-2888
jin-tonix.github.io
Yeoksam-dong, Gangnam-gu, Seoul

1. StyleSeller | AI & Automation PL

2025.11 ~ Present

On the business team, I automate work that staff used to repeat by hand: responding to partners, organizing records, calculating prices, and registering products. I watch how the work is done to decide what to automate, then own each system from requirements through design, implementation, deployment, and operations.

Project 1. Agent Platform and Skill Router

2026.05 ~ Present

Situation

With work split across multiple agents, context was lost at every handoff, and tracking and managing each agent took time. As shared skills grew to 115, having the LLM scan the full list to choose the right skill also drove up time and cost.

Action

- Designed and built the work-order and agent-delegation structure of an internal agent platform forked from Paperclip
- Capped delegation depth at 2 levels and delegations at 5 per work order
- Enforced tool-use approval in the server and DB rather than in prompts
- Designed a pipeline in which BM25 search first pulls relevant candidates from all skills, a judge model (Jev) checks gating rules and fit, and Gemini makes the final pick
- Built a 132-case evaluation set, found and fixed answer-key errors and fake skills, then re-measured

Result

- Routed 119 of 132 new questions correctly (90.15%)
- Reduced the skills Gemini reviews from 115 to 5
- Estimated Gemini call cost at about 1/10 of the previous level
- Rolled out the improved routing to all sub-agents
- Kept tool approvals from executing twice, even under 10 concurrent requests
- Enforced a daily cap of KRW 30,000 on paid API calls

Tech: Paperclip, Express, Drizzle, PostgreSQL, MCP SDK, Hermes, Python, FastAPI, LiteLLM, Redis, Langfuse, Gemini Flash

Project 2. Vendor CS Agent

2026.06 ~ Present

Situation

Partner inquiries arrived in staff members' KakaoTalk group chats, but with no official API, collecting conversations and responding automatically was hard. Replies also carry details that must not be wrong, such as amounts, commissions, and brand names, so sending LLM-generated answers as-is was not an option.

Action

- Built a Windows collector that captures KakaoTalk conversations on staff PCs and forwards them to the server
- Forked kakaocli to send replies from staff Macs and confirm delivery
- Matched chat room names against the active partner list; if no partner matches, nothing is generated or sent
- Generated reply drafts by searching partner records and the internal knowledge hub
- Built a verification step that checks amounts, rates, and brand names in each draft against source material and code, re-searching and revising on any mismatch
- Traced the causes of duplicate message collection and macOS send errors

Result

- Fixed a bug that collected one message 51 times, using a 15-second deduplication window

- Fixed a macOS Accessibility freeze that caused 31 of 147 sends (21%) to fail, using a 3-second timeout and delivery confirmation against the KakaoTalk DB
- In production, handling real partner inquiries
- 600 commits and 107 test files (as of 2026.09.23)

Tech: Swift, kakaocli, Python, LightRAG, Gemini Flash, Google Sheets, Google Drive

Project 3. Partner Status Board and Operations Ledger

Built 2026.10.19 ~ 2026.10.23, in production

Situation

Partner communication was scattered across KakaoTalk, the order mailbox, Slack, and Excel order sheets attached to emails. To see where each partner stood, staff had to check several channels by hand, and the same deal record often ended up in more than one place.

Action

- Designed the ledger data model and the matching rules that link records to partners
- Built an intake API and a mail collector that pull data from KakaoTalk, email, Slack, and Excel order sheets
- Built 6 views for tracking each partner's progress
- Summarized collected records by day and displayed them on each partner's calendar
- Matched records to partners without an LLM, in a fixed order: thread, business registration number, KakaoTalk room, then email
- Left records with conflicting or uncertain matching keys unclassified instead of auto-linking them, so staff can review them by hand

Result

- Displays the deal status of 400+ partners on a single screen
- Automatically collects about 100–200 deal records per weekday
- Built in 5 days and put into production
- Refreshes summaries for up to 500+ partners daily at 18:05
- Automatically halts the batch if daily AI cost exceeds KRW 2,000

Tech: React, Tailwind CSS, Vitest, Python, Pydantic, PostgreSQL, Gemini Flash

Project 4. Internal Work Tool and Pricing, Product, and Partner-Matching Automation

2026.03 ~ Present

Situation

Pricing depended on Google Sheets formulas, and staff handled two other tasks by hand: reviewing supplier catalogs to register products, and sending personalized KakaoTalk messages to sellers.

Action

- Ported the Google Sheets pricing formulas to code, matching the rounding units exactly
- Separated roles: the LLM returns only the input values to change, as JSON, and code performs the actual amount calculation
- Used the LLM to extract candidate product data from catalogs in PDF, image, Excel, and URL form
- Registered extracted products in the live catalog only after staff confirmation
- Sent personalized KakaoTalk messages to recipients passed from the CRM, routing any send to 500+ recipients to an approval queue
- Built workers that run partner and seller matching on staff PCs

Result

- Passed 43 price-calculation test cases
- Verified results for a sample of real products item by item against Google Sheets
- Sent personalized KakaoTalk messages to 261 people in one day (as of 2026.09.22)
- Deployed seller-search workers to 9 of 11 staff PCs, in production
- Replaced an authentication key shared across PCs with per-PC keys

Tech: React, Vitest, PostgreSQL, PostgREST, Gemini Flash, RAG, kakaocli

Project 5. LightRAG Knowledge Hub and Email Agent

2026.03.31 ~ Present

Situation

Several business agents needed to draw on the same internal company information. Partner inquiries arriving in the shared mailbox also needed reply drafts grounded in real business documents, not just generated text.

Action

- Designed a workspace registry and knowledge search structure shared by multiple agents
- Implemented a search-planning, evidence-checking, and draft-verification sequence
- Built an email agent that searches internal knowledge and partner materials for each inquiry and creates a reply draft in the assignee's Gmail drafts folder
- Implemented verification logic that checks up to 6 claims in each generated reply against source evidence
- Designed the flow so unsupported claims trigger a rewrite, and the draft is discarded entirely if unsupported content remains after the rewrite
- Kept sending manual, with the assigned staff member making the final check

Result

- 4 agents share knowledge across 13 workspaces
- Added validation so the server will not start if a workspace name is linked incorrectly
- Fixed a cursor-saving bug that had dropped 48 of 1,943 sent emails from the learning data
- Staff now check the evidence before sending each reply

Tech: Python, FastAPI, Pydantic, LightRAG, PostgreSQL, pgvector, Neo4j, FlashRank, Gemini Flash, Gmail, pytest

Project 6. Content Autopilot

2026.10.03 ~ Present

Situation

About a year earlier I had tried to automate content production, but the video generation models of the time fell short on quality, so I settled for semi-automation. As the models improved, I went back to full automation.

Action

- Built a flow in which a staff request for a Reel in the internal work tool leads to script generation, video rendering, and preview approval
- Built 9:16 MP4 generation and saving to Google Drive
- Designed a state machine, not an LLM, to manage progress through each production stage
- Configured defaults: manual approval for publishing, dry-run for recurring automation, and paid generation disabled
- Automatically attached an AI-generated content label to generated content
- Separated pre-publish review from approval

Result

- Completed a production canary that generates a 1080×1920 MP4 and saves it to Drive
- The new video path is still in prompt tuning and has not posted to social media yet
- A separate path that applies per-account captions and hashtags to existing videos and posts them to multiple accounts at once is live in production
- Enforced a limit of one post per account per day with a DB index
- 617 test files in total across 3 related services

Tech: Python, Gemini Flash, Hermes, HyperFrames, ffprobe, Google Drive, Supabase, PostgreSQL

Project 7. Staff PC Deployment and Operations Environment

2026.02 ~ Present

Situation

Automation tasks such as KakaoTalk collection, Instagram posting, and seller matching had to run on staff Macs and Windows PCs. Setting up each PC by hand made updates and authentication hard to keep track of, and the load grew with every new hire.

Action

- Built Mac and Windows installers that run with a single command
- Designed per-device authentication using one-time tokens and individually issued keys
- Built auto-updates with update timing staggered per PC
- Added a safeguard: the server deploys only after confirming a committed image exists, and halts the deploy otherwise
- Set up a cron job that reports service anomalies to Slack
- Built a fallback server and restored service when a cloud DB usage limit took down the work-tool API

Result

- Deployed to 11 staff PCs (Mac and Windows) with a one-line install
- When a cloud DB usage limit took down the work-tool API on 2026.08.20, stood up an identical PostgREST API on a VM and restored service the same day
- Restored service without modifying the roughly 100 DB call sites in the app
- Migrated 4 DBs to self-hosting between 2026.08.06 and 08.25

Tech: macOS Bash, Windows PowerShell, launchd, Windows Task Scheduler, Node.js, Python, PostgreSQL, PostgREST, NGINX, Docker, GCP, Slack

2. We:Review | AI Development Intern

2025.08 ~ 2025.10

Project 8. Receipt Review Automation Pipeline

Situation

Staff extracted and checked product names and prices from receipt images by hand. Long receipts and scattered text made raw OCR output hard to use, and re-checking products and prices took a lot of time.

Action

- Split receipt images into tiles and extracted text with PaddleOCR
- Reconstructed lines from scattered OCR output with DBSCAN
- Extracted product names and prices as structured JSON using few-shot prompting on an open Qwen-family LLM
- Turned the review pipeline into a service with Python and FastAPI

Result

- Raised review accuracy from 70% to 95%
- Reduced review time by 85%, based on my own measurements
- Contributed to building and launching a review automation pipeline that runs from OCR through LLM-based structured data extraction

Tech: Python, PaddleOCR, DBSCAN, Qwen-family LLM, FastAPI

3. Previous Experience

Jeonwon Daycare Center & After-School Program | Operations Manager 2015.03 ~ 2018.02

Hired teachers, planned the annual curriculum and events, communicated with parents and the operating committee, and managed operations on behalf of the director.

Jeongsang Language School · Pots&Pans · Montekids English Academy | English Teacher 2018.08 ~ 2024.02

Designed lessons, created teaching materials, built individual learning plans, and handled parent consultations and learning management.